

보고서

1. 문제 상황

기존의 결합 해제된 아키텍처 구성을 통해 Web과 Application 기능을 분리하여 서비스 간 종속성을 완화하였다.

그러나 관리자는 보다 안전하고 확장성 있는 애플리케이션 환경을 구축하기 위해 추가적인 개선이 필요하다고 판단하였다.

특히 고객에게 서비스를 제공하는 비즈니스 애플리케이션은 사용량에 따라 트래픽이 증가하거나 감소할 수 있으므로, 안정적인 서비스 제공을 위해서는 트래픽을 여러 서버로 분산하고 필요에 따라 서버의 수를 자동으로 조절할 수 있는 구조가 필요하다.

또한 데이터베이스 장애가 발생하더라도 서비스의 연속성을 유지할 수 있도록 데이터베이스의 고가용성 구성도 필요하다.

따라서 이번 과제에서는 기존의 결합 해제된 Web/Application 구조를 기반으로 ELB, EC2 Auto Scaling, RDS Multi-AZ를 적용하여 고가용성과 확장성을 강화한 아키텍처를 설계한다.

2. 학습 목표

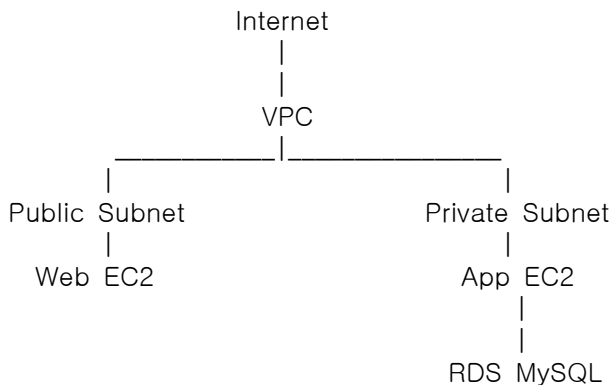
이번 과제를 통해 다음 내용을 이해하는 것을 목표로 한다.

- ELB를 통한 서비스 요청 분산 및 안정적인 애플리케이션 서비스 제공
- EC2 Auto Scaling을 통한 수요에 따른 서버 규모 조정
- RDS Multi-AZ를 통한 데이터베이스 고가용성 확보
- 기존에 학습한 VPC, Public/Private Subnet, EC2, RDS를 고가용성 아키텍처에 적용
- AWS Well-Architected Framework의 신뢰성 및 성능 효율성 관점 이해

3. 기존 아키텍처

이전 과제에서는 하나의 EC2 인스턴스에 Web과 Application 기능이 함께 존재하던 구조에서 벗어나 Web과 Application을 분리하였다.

기존 결합 해제 구조는 다음과 같다.



이 구조를 통해 Web과 Application의 기능을 분리할 수 있었지만, 각 계층에 단일 EC2가 존재하기 때문에 다음과 같은 한계가 있다.

- Web 서버 장애 발생 시 Web 서비스 이용이 어려움
- Application 서버 장애 발생 시 애플리케이션 기능 이용이 어려움
- 트래픽 증가 시 서버를 수동으로 추가해야 함
- 트래픽 감소 시 사용하지 않는 서버 자원이 남을 수 있음
- 단일 DB 구성에서는 데이터베이스 장애에 대한 대응이 제한적임

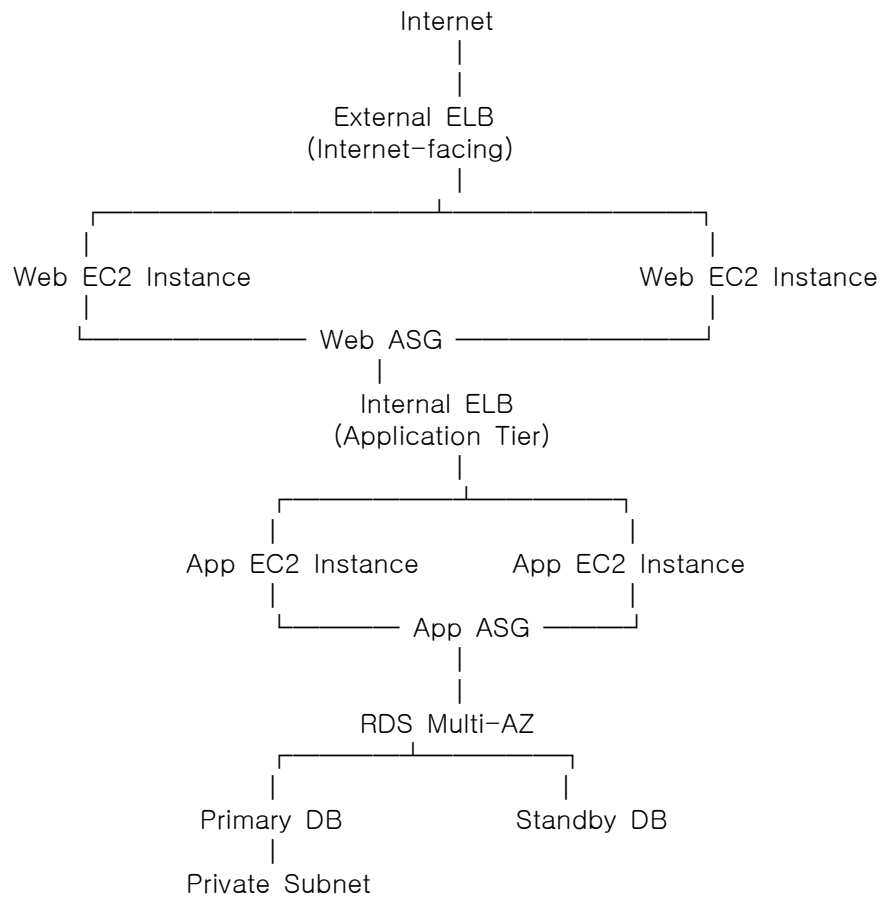
따라서 고가용성과 자동 확장 기능을 추가할 필요가 있다.

4. 개선 아키텍처

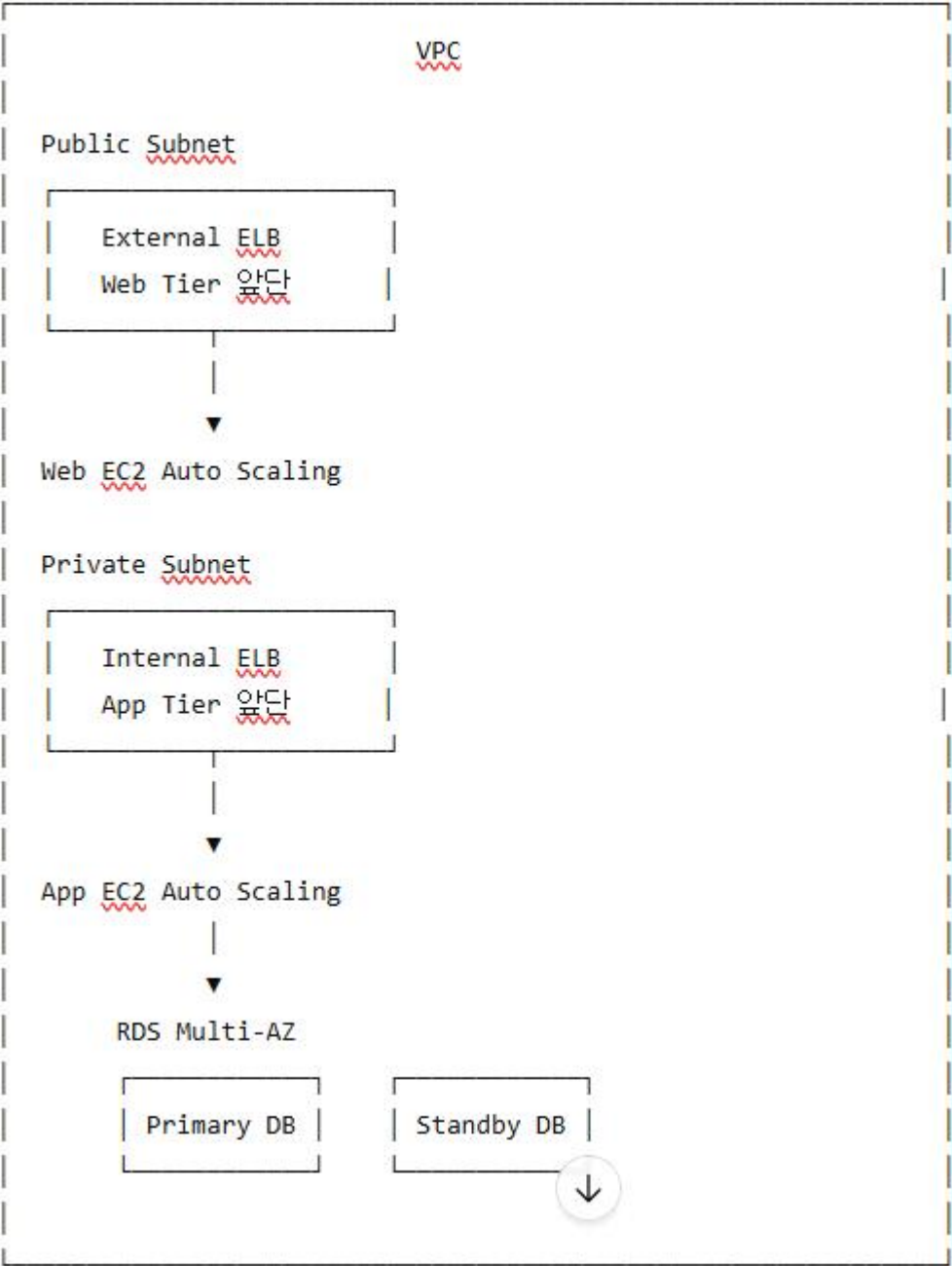
기존 아키텍처에 다음 기능을 추가하여 고가용성 및 확장성을 확보한다.

1. 외부 ELB를 Web Tier 앞에 배치
2. Web Tier 뒤에 EC2 Auto Scaling 구성
3. 내부 ELB를 Application Tier 앞에 배치
4. Application Tier 뒤에 EC2 Auto Scaling 구성
5. RDS Multi-AZ를 이용하여 데이터베이스 고가용성 확보

최종 아키텍처



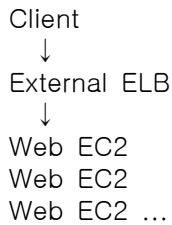
네트워크 구성 개념



5. 외부 ELB 구성

인터넷에서 들어오는 사용자의 요청을 Web Tier의 여러 EC2 인스턴스로 분산하기 위해 외부 ELB를 구성한다.

역할



사용자의 요청을 하나의 Web 서버에 집중시키지 않고 여러 서버에 분산할 수 있다. 또한 ELB는 등록된 대상의 상태를 확인하여 정상적으로 동작하는 서버로 요청을 전달하도록 구성할 수 있다.

기대 효과

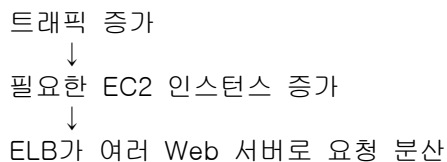
- Web 서버의 부하 분산
- 특정 Web 서버 장애 발생 시 다른 서버로 요청 전달
- 여러 Web 서버를 이용한 고가용성 확보
- Web Tier의 확장 기반 마련

6. Web Tier EC2 Auto Scaling

Web Tier에는 EC2 Auto Scaling을 적용한다.

Auto Scaling은 애플리케이션의 수요와 설정된 조건에 따라 EC2 인스턴스의 수를 자동으로 조절할 수 있도록 한다.

동작 개념



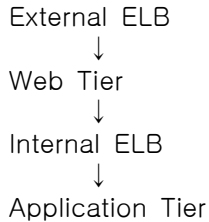
반대로 트래픽이 감소하면 필요하지 않은 인스턴스를 줄여 불필요한 리소스 사용을 감소시킬 수 있다.

기대 효과

- 수요 증가에 따른 자동 확장
- 수요 감소에 따른 자동 축소
- 서버 자원의 효율적인 사용
- 수동 서버 관리 부담 감소

7. 내부 ELB 구성

Application Tier 앞에는 외부에 직접 노출되지 않는 내부 ELB를 구성한다.
구조는 다음과 같다.



내부 ELB를 사용하면 Web Tier에서 Application Tier로 전달되는 요청을 여러 Application EC2 인스턴스에 분산할 수 있다.

기대 효과

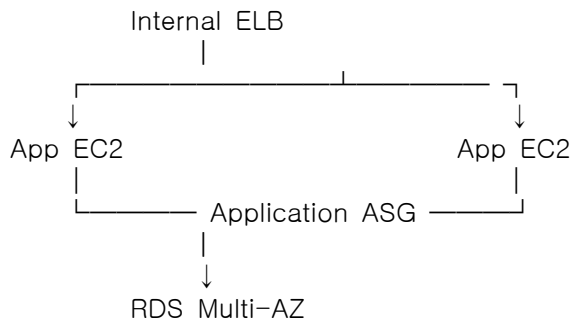
- Application 서버 부하 분산
- Application 서버 장애 대응
- Application Tier의 확장성 확보
- Web Tier와 Application Tier 간 역할 분리 강화

특히 Application 서버를 외부 인터넷에 직접 노출하지 않고 내부 ELB를 통해 접근하도록 구성하면 계층별 접근 범위를 분리하는 데 도움이 된다.

8. Application Tier EC2 Auto Scaling

Application Tier에도 EC2 Auto Scaling을 적용한다.

Application 서버의 처리량이 증가하면 EC2 인스턴스를 추가하고, 사용량이 감소하면 불필요한 인스턴스를 줄이는 방식으로 운영할 수 있다.

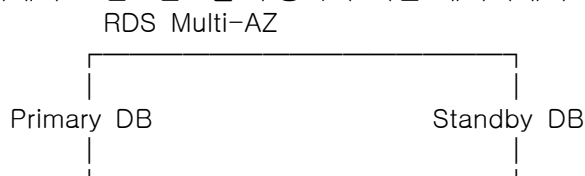


이를 통해 Web Tier와 Application Tier를 각각 독립적으로 확장할 수 있다.

9. RDS Multi-AZ 구성

데이터베이스는 단일 RDS 구성에서 RDS Multi-AZ 구성으로 개선한다.

RDS Multi-AZ는 데이터베이스의고가용성을 확보하기 위한 구성으로, 서로 다른 가용 영역에 데이터베이스 인스턴스를 구성하여 기본 데이터베이스 장애에 대비한다.



Primary DB에 장애가 발생하면 AWS의 장애 조치 기능을 통해 Standby DB를 활용하여 데

데이터베이스 서비스의 가용성을 높일 수 있다.

기대 효과

- 데이터베이스 장애에 대한 대응
- 서비스 중단 시간 감소
- 데이터베이스 고가용성 향상
- 여러 Availability Zone을 활용한 장애 대응

RDS Multi-AZ의 Standby DB는 일반적인 읽기 트래픽을 처리하기 위한 Read Replica와 목적이 다르며, 이번 과제에서는 Primary DB 장애에 대비한 고가용성 확보가 핵심이다.

10. 최종 데이터 흐름

최종적으로 사용자의 요청은 다음과 같은 흐름으로 처리된다.

사용자



Internet



External ELB



Web EC2 Auto Scaling



Internal ELB



Application EC2 Auto Scaling



RDS Multi-AZ



Database

각 계층의 역할을 분리함으로써 하나의 서버에 모든 기능을 집중시키는 구조에서 벗어나 확장성과 장애 대응력을 높일 수 있다.

11. ELB와 EC2 Auto Scaling의 장점

ELB

ELB의 주요 목적은 여러 서버에 요청을 분산하는 것이다.

- 트래픽 분산
- 서버 장애 대응
- 서비스 가용성 향상
- 여러 EC2 인스턴스를 하나의 서비스 엔드포인트로 연결

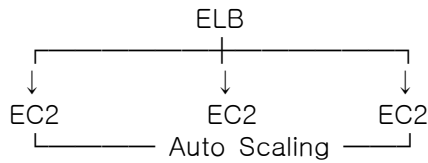
EC2 Auto Scaling

Auto Scaling은 EC2 인스턴스의 수를 수요에 맞게 조정하는 역할을 한다.

- 트래픽 증가 시 확장
- 트래픽 감소 시 축소
- 리소스 사용 효율 향상
- 수동 서버 관리 부담 감소
- 변화하는 서비스 수요에 대한 대응

두 서비스의 결합

ELB와 Auto Scaling을 함께 사용하면 다음과 같은 구조를 만들 수 있다.



Auto Scaling이 EC2 인스턴스 수를 조절하고, ELB가 현재 실행 중인 인스턴스에 요청을 분산한다.
따라서 서비스의 수요 변화에 유연하게 대응하면서도 안정적인 서비스를 제공할 수 있다.

12. 기존 아키텍처와 최종 아키텍처 비교

구분	기존 구조	최종 개선 구조
Web	단일 EC2	External ELB + EC2 Auto Scaling
Application	단일 EC2	Internal ELB + EC2 Auto Scaling
Database	단일 RDS	RDS Multi-AZ
트래픽 분산	제한적	ELB를 통한 분산
서버 확장	수동	Auto Scaling
서버 장애 대응	제한적	ELB를 통한 정상 서버 연결
DB 장애 대응	제한적	Multi-AZ 구성
가용성	상대적으로 낮음	항상
확장성	제한적	항상
운영 효율성	수동 관리 비중 높음	자동화 기능 활용

13. Well-Architected Framework 관점

이번 설계는 AWS Well-Architected Framework의 여러 원칙과 연관된다.

Reliability

가장 직접적으로 적용되는 영역이다.

- ELB를 통한 장애 서버 제외 및 트래픽 분산
- EC2 Auto Scaling을 통한 서버 수 조절
- RDS Multi-AZ를 통한 DB 장애 대응

이를 통해 개별 구성 요소의 장애가 전체 서비스 장애로 이어지는 것을 줄일 수 있다.

Performance Efficiency

서비스의 사용량에 맞춰 EC2 리소스를 조절할 수 있다.

Auto Scaling을 통해 수요가 증가하면 서버를 확장하고, 수요가 감소하면 서버를 축소하여 변화하는 트래픽에 효율적으로 대응할 수 있다.

Operational Excellence

서버 수를 수동으로 조절하는 대신 Auto Scaling을 활용하고, ELB를 통해 트래픽 분산을 자동화함으로써 운영 관리 부담을 줄일 수 있다.

Security

Web과 Application 계층을 분리하고 Application 및 Database 계층을 Private Subnet에 배치하여 외부에 직접 노출되는 영역을 최소화하는 방향으로 설계할 수 있다.

14. 아키텍처 개선 효과

이번 개선을 통해 다음과 같은 효과를 얻을 수 있다.

1. 고가용성

ELB와 여러 EC2 인스턴스를 이용하여 특정 서버의 장애가 전체 서비스 장애로 이어지는 것을 줄인다.

2. 확장성

EC2 Auto Scaling을 이용하여 서비스 수요에 따라 서버의 규모를 자동으로 조정할 수 있다.

3. 데이터베이스 가용성

RDS Multi-AZ를 적용하여 Primary DB 장애에 대비하고 데이터베이스 서비스의 가용성을 향상시킨다.

4. 장애 대응력 향상

Web, Application, Database 각 계층에 장애 대응 구조를 적용하여 전체 서비스의 안정성을 높인다.

5. 운영 효율성 향상

트래픽 분산과 서버 규모 조정을 자동화하여 관리자의 수동 운영 부담을 줄인다.

15. 결론

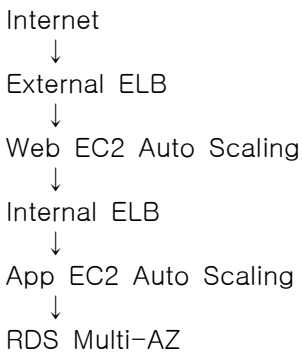
이번 과제에서는 기존의 결함 해제된 Web, Application, Database 구조를 기반으로 고가용성과 확장성을 강화하는 아키텍처를 설계하였다.

Web Tier 앞에는 외부 ELB를 구성하고 Web EC2에 Auto Scaling을 적용하여 사용자 요청을 여러 서버에 분산하고 수요에 따라 서버 규모를 조정할 수 있도록 하였다.

Application Tier 앞에는 내부 ELB를 구성하고 Application EC2에도 Auto Scaling을 적용하여 Application 서버의 부하를 분산하고 독립적인 확장이 가능하도록 하였다.

또한 데이터베이스에는 RDS Multi-AZ를 적용하여 Primary DB 장애에 대비한 Standby DB 구성을 통해 데이터베이스의 가용성을 향상시켰다.

최종적으로 다음과 같은 고가용성 구조를 완성하였다.



이를 통해 단일 서버에 의존하는 구조에서 벗어나 트래픽 분산, 자동 확장, 데이터베이스 장애 대응이 가능한 안정적이고 확장성 있는 AWS 아키텍처를 설계하였다.

이번 과제를 통해 ELB, EC2 Auto Scaling, RDS Multi-AZ가 각각 독립적으로 동작하는 기능이 아니라, 서로 결합하여 AWS에서 요구되는 고가용성, 확장성, 안정성 및 운영 효율성을 구현하는 핵심 구성 요소라는 것을 이해하였다.